

Cache Simulator Report

1 Implementation Details

The cache simulator has been implemented in C++ using a modular and object-oriented approach. The main components are:

1. Main Driver: `main.cpp`

This file initializes the simulation environment and drives the application by:

- Reading configuration and trace files.
- Instantiating cache controllers.
- Managing the simulation loop.

2. Cache Access: `cacheAccess.cpp/h`

This module defines the logic to access individual cache blocks, perform tag checks, determine hit/miss, and handle block replacements using LRU policies.

3. Cache Controller: `cacheController.cpp/h`

Handles operations at the level of the whole cache:

- Coordinates between memory requests and cache responses.
- Implements cache coherence if necessary.
- Maintains statistics (hits, misses, execution time).

4. Cache Structures: `cacheStructure.h`

Defines the internal data representation of cache blocks and sets. Important structures include:

- `CacheBlock` with tag, valid/dirty bits.
- `CacheSet` with a list/vector of blocks, supporting LRU management.

5. Bus Operation: `busOperation.cpp/h`

Responsible for coordinating memory access across processors:

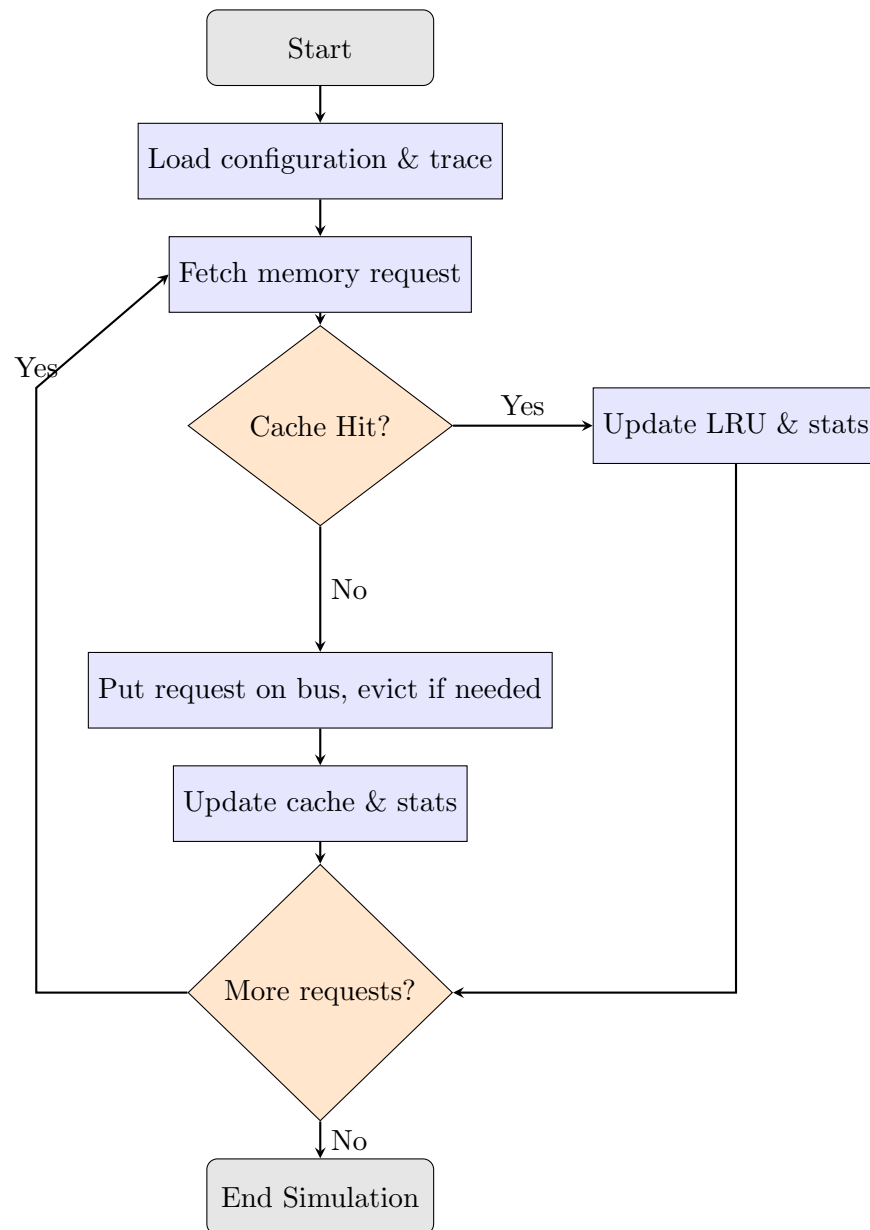
- Emulates memory bus or interconnect.
- Synchronizes memory operations (loads/stores).

6. Configuration: `cacheconf.h`

Defines simulation parameters such as:

- Cache size (default: 4KB).
- Associativity (default: 2-way).
- Block size (default: 32 bytes).

Flowchart of Cache Simulation



2 Cache Coherence

To ensure consistent memory views across the four L1 caches in our quad-core simulator, we implemented cache coherence using the MESI protocol within a bus-based snooping architecture. The following outlines the coherence model and detailed behavior under different memory access cases.

- **Bus-Based Snooping:** Each cache monitors a shared global bus for memory operations initiated by other processors. This allows each cache to snoop and respond to relevant transactions to maintain coherence.
- **Write Invalidate Protocol:** We adopt a write-invalidate approach, where a processor writing to a cache line broadcasts an invalidate message, causing other caches to invalidate their copies. This significantly reduces bus bandwidth usage compared to write-update schemes, particularly for high-write traffic scenarios.
- **MESI States:**
 - **Modified (M):** Cache line is only in the local cache and has been modified.
 - **Exclusive (E):** Cache line is clean and held exclusively by one cache.
 - **Shared (S):** Cache line is clean and may exist in multiple caches.
 - **Invalid (I):** Cache line is invalid or out-of-date.
- **Assumptions of MESI protocol followed**
 - **Read Hit:**
 - * No bus activity is triggered.
 - * Data is simply returned from the local cache.
 - * State remains unchanged (must be in M, E, or S).
 - * In all the cases only 1 cycle will be required as we are directly getting hits in the L1 cache.
 - **Read Miss:**
 - * Bus read is issued. Action depends on other cache states:
 - *No copies elsewhere:* Data is read from memory and stored with state **E**. (100 cycles of memory penalty)
 - *One copy in E:* That cache supplies data over the bus (cache to cache transfer); both change to **S**. (2*N cycles of transfer penalty)
 - *Copies in S:* One cache supplies data (cache to cache transfer); all caches including requester stay in **S**. (2*N cycles of transfer penalty)
 - *Copy in M:* M cache writes back data to memory and shares it on the bus (cache to cache transfer); source transitions M → S, requester gets **S**. (100 cycles of cache penalty + 2*N cycles of transfer penalty)
 - In all the above cases an additional 1 cycle will be required for cache access.
 - **Write Hit:**
 - * *M state:* Line is already dirty. Update value, no state change. (1 cycle for direct hit)
 - * *E state:* Line is clean and private. Update value, state changes to **M**. (1 cycle for direct hit)

- * *S state*: Broadcast invalidate. Other caches with S copy move to **I**. Update local value, local state becomes **M**.(1 cycle for hit ,we are doing invalidation in same cycle as the miss happened at local cache)
- **Write Miss**:
 - * Bus transaction issued as RWITM (Read With Intent To Modify).
 - * Depending on copies in other caches:
 - *No copies*: Data is read from memory, updated, and marked (100 cycles for memory access)**M**.
 - *Copies in S or E*: Invalidate broadcast forces others to **I**. Local line is updated and marked **M**.(100 cycles for memory access)
 - *Copy in M*: The owner (in M) delays RWITM, writes back to memory, and invalidates its line. Then the requester retries RWITM, updates its line, and sets it to **M**.(100 cycles for writing copy to memory and 100 cycles again for reading value from memory into local cache)
- **Remote Processor Accesses (Snooped Events)**
 - **Mem Read**:
 - * Caches with line in E, S, or M may respond.
 - * M state cache writes back value, transitions to **S**.
 - * E or S caches stay in **S**.
 - **RWITM (Write Miss)**:
 - * All caches holding the line invalidate their copy (S or E $\rightarrow$ I).
 - * M cache must write back and then invalidate (M $\rightarrow$ I).
 - **Invalidate**:
 - * All caches matching the address invalidate their line.
- **Write-Back Support**: In contrast to write-through caches, we use write-back policy. Modified lines are written to memory only on eviction, reducing bus traffic and improving performance.

3 Assumptions and Interpretations

1. Bus arbitration

The bus arbitration is assumed as follows: Ties are broken arbitrarily on the bus as is done in real systems when multiple cores attempt bus transactions simultaneously. The operating system's thread scheduler determines which waiting thread gets the bus access when multiple cores try to access it (we started the project earlier so we have taken this assumption as was written originally in the pdf) So we were able to obtain different data across simulation which we could analyse properly.

2. Bus Transactions

Only one active bus transaction can happen at any time, including memory fetches, cache-to-cache transfers, and invalidates. Local Cache hit 1 cycles, if its miss the request gets placed on bus, we are doing invalidation/RDx/WRX also in same cycle as the miss happened at local cache.

4 Definitions

1. Core Statistics

Based on the implementation in the source code, the following statistics are tracked and calculated for each core:

2. Total Instructions

The total number of memory access operations (reads and writes) processed by the cache controller. Each memory access from the trace file increments `totalInstructions` counter in the `cacheController.cpp` after the `cacheAccess` function completes execution.

3. Total Reads

A counter that specifically tracks read operations. This is incremented in the `runTracefile` method when a line starting with "R" is encountered in the trace file, and also updated in `cacheAccess` when `isWrite` is false.

4. Total Writes

Calculated as the difference between total instructions and total reads: `totalInstructions - totalReads`. This represents all write operations processed by the cache.

5. Total Execution Cycles

The `globalCycles` counter counts cycles spent executing its own instruction trace.

- Cache access: Always adds `config.cacheAccessCycles` (100 cycles)
- Cache misses: Adds additional memory access cycles
- Bus operations: Adds cycles depending on operations like invalidations and state transitions
- MESI transitions: Different state changes have different cycle penalties

6. Idle Cycles

Idle cycles are cycles spent waiting for the resources while other cores are executing its instructions. Once a core finishes executing all its instructions, it is considered done. Further cycles are not counted under idle cycles for that core.

7. Cache Misses

The `globalMisses` counter is incremented in `cacheAccess` when `response->hit` is 0, indicating either:

- The tag wasn't found in the cache
- The block was found but marked invalid

8. Cache Miss Rate

Calculated as $(\text{globalMisses} / \text{totalInstructions}) * 100$, giving a percentage of how often memory accesses missed the cache.

9. Cache Evictions

The `globalEvictions` counter tracks when existing blocks must be removed to make room for new blocks. This is incremented in `cacheAccess` when `response->eviction` is not 0.

10. Writebacks

- Writebacks are incremented whenever any cache writes back to memory nothing else is added in writebacks.

The total of a core represents all memory writes from a cache to main memory.

11. Bus Invalidations

The `busInvalidations` counter is incremented in `busOperation.cpp` when another core forces this core to invalidate a cache line through bus signaling. This happens when:

- Another core writes to a shared line
- Cache coherence protocol requires invalidation for consistency

12. Data Traffic (Bytes)

The `dataTrafficBytes` counter represents memory bandwidth usage, incremented by `blockSize` when:

- Cache misses occur (reading from memory)
- Dirty evictions occur (writing back to memory)
- Cache-to-cache transfers occur (particularly for shared data)
- MESI transitions that require data movement

13. Overall Bus Summary

a. Total Bus Transactions

Represents the total number of operations that occurred on the bus across all cores.

b. Total Bus Traffic (Bytes)

The sum of `dataTrafficBytes` from all cores, representing the total amount of data transferred over the bus during the simulation.

5 Experimental Results

To understand the behavior and consistency of our cache simulator, we conducted multiple experimental runs with identical parameters. Each experiment used the default configuration of a 4KB 2-way set associative L1 cache per processor with 32-byte block size. The simulator was run 10 times with the same application input, and the results were collected to analyze the variability in performance metrics.

5.1 Simulation Parameters

- **Cache Size:** 4KB per processor
- **Associativity:** 2-way set associative
- **Block Size:** 32 bytes
- **Number of Cores:** 4
- **Number of Simulation Runs:** 10

5.2 Variation in Execution Cycles

Figure 1 shows the distribution of execution cycles across all 10 runs for each of the four cores.

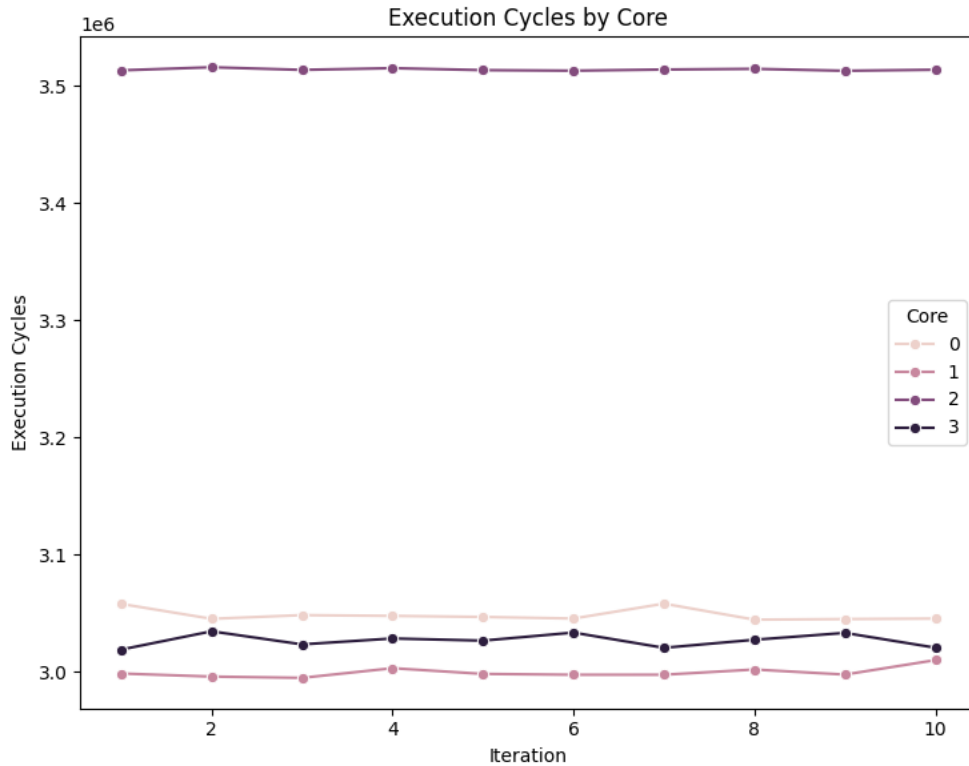


Figure 1: Execution Cycles Distribution Across 10 Runs for Each Core

The execution cycles data reveals several important observations:

- **Core-specific Patterns:** Core 2 consistently requires significantly more execution cycles (approximately 3.51 million) compared to the other cores, which average between 3.0-3.05 million cycles. This indicates that Core 2 is likely executing more memory-intensive operations or experiencing more complex memory access patterns.
- **Run-to-Run Variation:** Despite identical input parameters, we observe minor variations in execution cycles across different runs. For example, Core 0's execution cycles range from approximately 3,044,417 to 3,058,073 cycles, showing a variation of about 0.45%.
- **Relative Consistency:** While absolute cycle counts vary, the relative performance between cores remains consistent across runs, with Core 2 always requiring the most cycles, followed typically by Core 0, then Core 3, and Core 1 generally being the most efficient.

5.3 Analysis of Cache Miss Rates

Figure 2 illustrates the miss rates for each core across all 10 simulation runs.

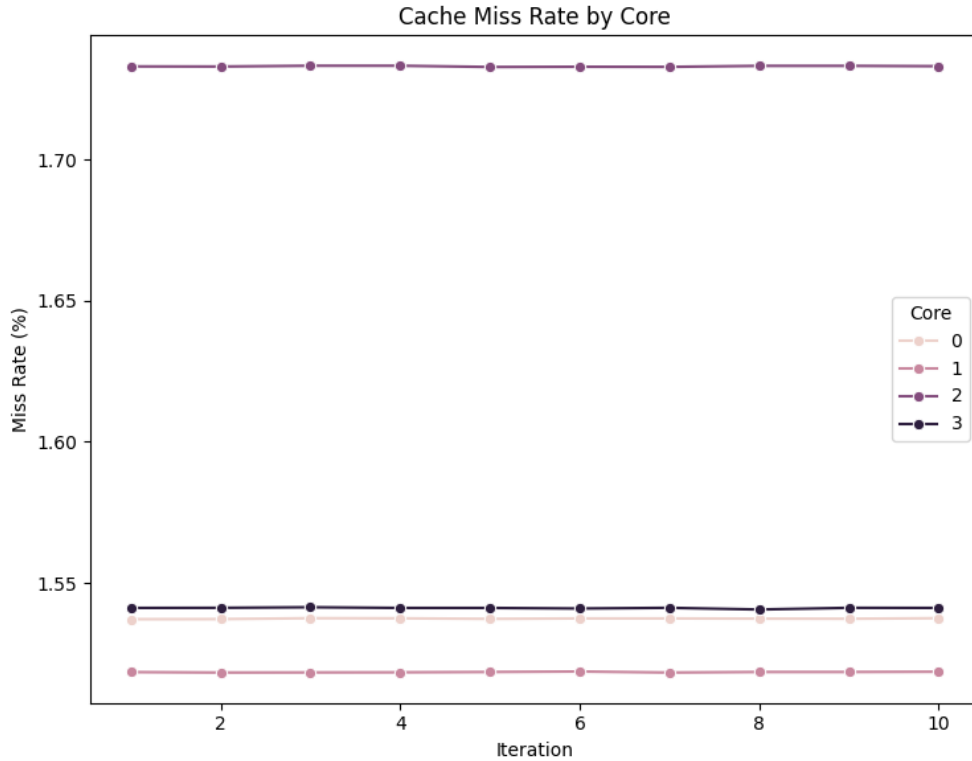


Figure 2: Cache Miss Rate Distribution Across 10 Runs for Each Core

The miss rate data reveals interesting patterns:

- **Remarkable Stability:** Unlike execution cycles, miss rates demonstrate exceptional stability across different runs for each individual core. For instance, Core 0's miss rate remains within the narrow range of 1.53715% to 1.53751%.
- **Core-specific Miss Behaviors:** Core 2 consistently shows the highest miss rate (approximately 1.733%), which aligns with its higher execution cycle count. Meanwhile, Core 1

exhibits the lowest miss rate (approximately 1.518%), explaining its generally lower execution cycles.

- **Correlation with Performance:** The hierarchy of miss rates (Core 2 $\hat{>}$ Core 0 $\hat{>}$ Core 3 $\hat{>}$ Core 1) strongly correlates with the general pattern observed in execution cycles, confirming that cache performance is a dominant factor in overall execution efficiency.

6 Cache Parameter Impact Analysis

To investigate the performance impact of different cache parameters, we extended the simulator to compute maximum execution time across all cores for various cache configurations. We systematically varied three key parameters (cache size, associativity, and block size) from the default configuration to three alternative values in powers of 2, keeping other parameters constant during each analysis.

6.1 Impact of Number of Sets

Figure 3 illustrates the maximum execution time across cores as we vary the number of cache sets from 32 to 128, effectively changing the cache size while keeping associativity and block size constant.

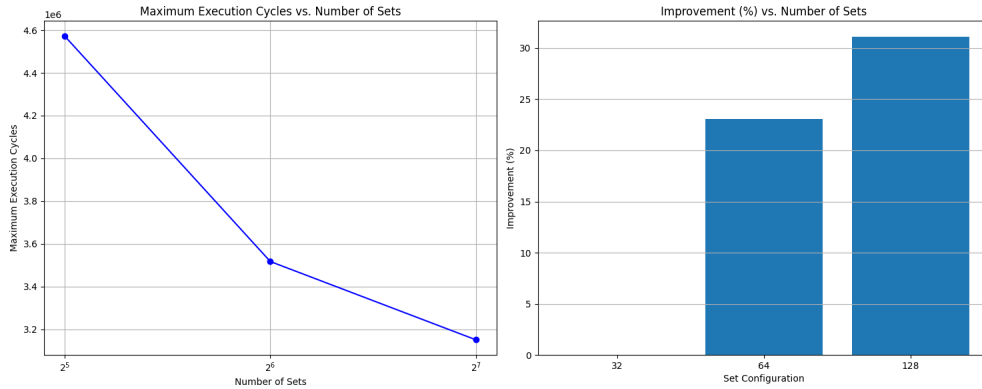


Figure 3: Maximum Execution Time vs. Number of Sets

As the number of sets increases from 32 to 128 (effectively increasing cache size from 2KB to 8KB with constant associativity of 2 and block size of 32 bytes), we observe a dramatic reduction in execution time. The most significant improvement occurs when doubling the sets from 32 to 64, yielding a 23.08% reduction in execution cycles. Further doubling to 128 sets provides an additional improvement, but with diminishing returns, reaching a total reduction of 31.10% compared to the baseline.

This behavior can be attributed to a significant reduction in conflict misses as addresses are distributed across more sets. The diminishing returns at higher set counts suggest that beyond a certain point, the workload’s working set fits more comfortably in the cache, and remaining misses are likely compulsory or capacity misses rather than conflict misses.

6.2 Impact of Associativity

Figure 4 shows how maximum execution time changes as we increase associativity from 2-way to 8-way, keeping the number of sets and block size constant.

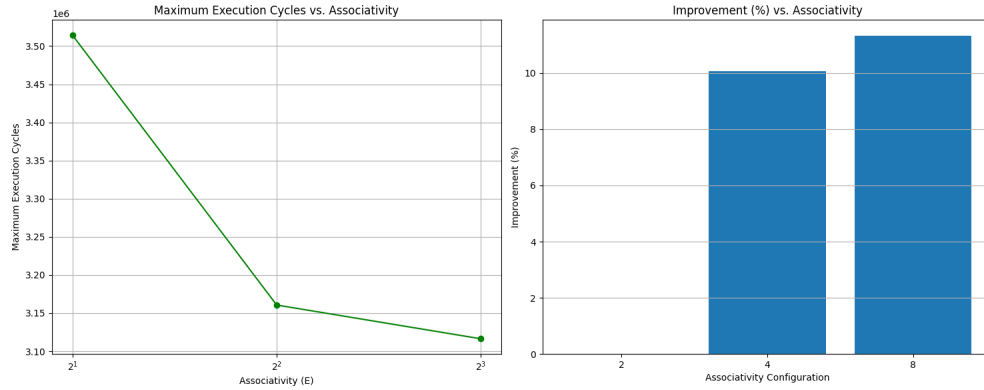


Figure 4: Maximum Execution Time vs. Associativity

Increasing associativity from 2-way to 4-way yields a substantial 10.07% improvement in execution time. However, further increasing to 8-way provides only a marginal additional benefit of about 1.26%, for a total improvement of 11.33% compared to the baseline.

This pattern indicates that the workload experiences a significant number of conflict misses that can be mitigated by increasing associativity from 2 to 4. The minimal additional gain from 4-way to 8-way suggests that after 4-way, most conflict misses have been eliminated, and remaining misses are likely compulsory or capacity misses that cannot be addressed by increasing associativity alone.

6.3 Impact of Block Size

Figure 5 demonstrates the effect of varying block size from 16 bytes to 64 bytes on maximum execution time.

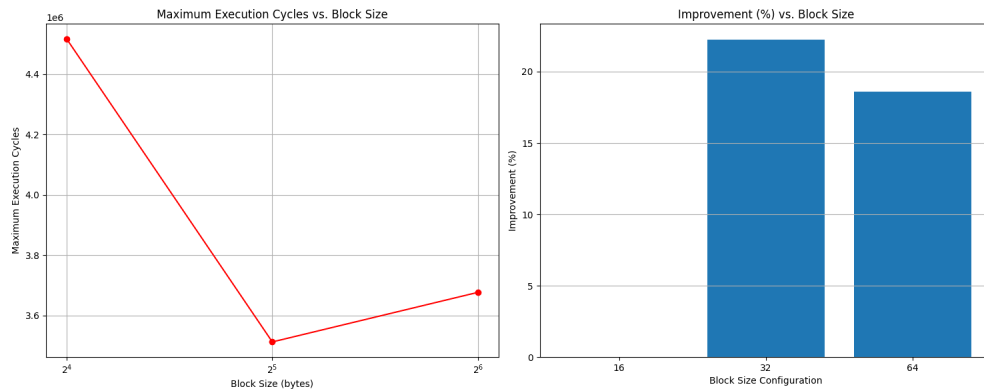


Figure 5: Maximum Execution Time vs. Block Size

The block size experiments reveal a more complex relationship with performance. Increasing block size from 16 bytes to 32 bytes results in a substantial 22.24% reduction in execution time, indicating strong spatial locality in the workload. However, further increasing to 64 bytes causes

performance to degrade slightly, with execution time increasing by 4.68% compared to the 32-byte configuration.

This non-monotonic behavior suggests an interesting tradeoff: while larger blocks can better exploit spatial locality by prefetching nearby data, excessively large blocks can lead to cache pollution and increased miss penalties when spatial locality is not uniformly strong. The 32-byte block size appears optimal for this workload, balancing the benefits of prefetching against the costs of transferring unnecessarily large blocks.

6.4 Overall Parameter Impact Analysis

Comparing across all three parameters, we observe that:

- **Number of sets** (effectively cache size) has the largest overall impact, with a maximum improvement of 31.10% when increased from 32 to 128 sets.
- **Block size** shows the second largest impact but with an optimal point at 32 bytes, demonstrating a 22.24% improvement over 16 bytes.
- **Associativity** provides more modest but still significant benefits of 11.33% when increased from 2-way to 8-way.

The optimal cache configuration for this workload appears to be 128 sets with 4-way associativity and 32-byte blocks. This configuration strikes a good balance between reducing conflict misses through increased associativity and sets while optimizing for the workload’s spatial locality characteristics with an appropriate block size.

These findings emphasize the importance of tuning cache parameters to specific workload characteristics, as each parameter affects different types of cache misses. For this particular application, increasing the number of sets (effectively the cache size) provides the most substantial performance benefit, suggesting that the working set size is a primary factor limiting performance.